

AXI_SPI_I2C

대한상공회의소 최은수

Contents

Chapter 1. **AXI_Introduction**

Chapter 2. **AXI_UVM**

Chapter 3. **AXI_C**

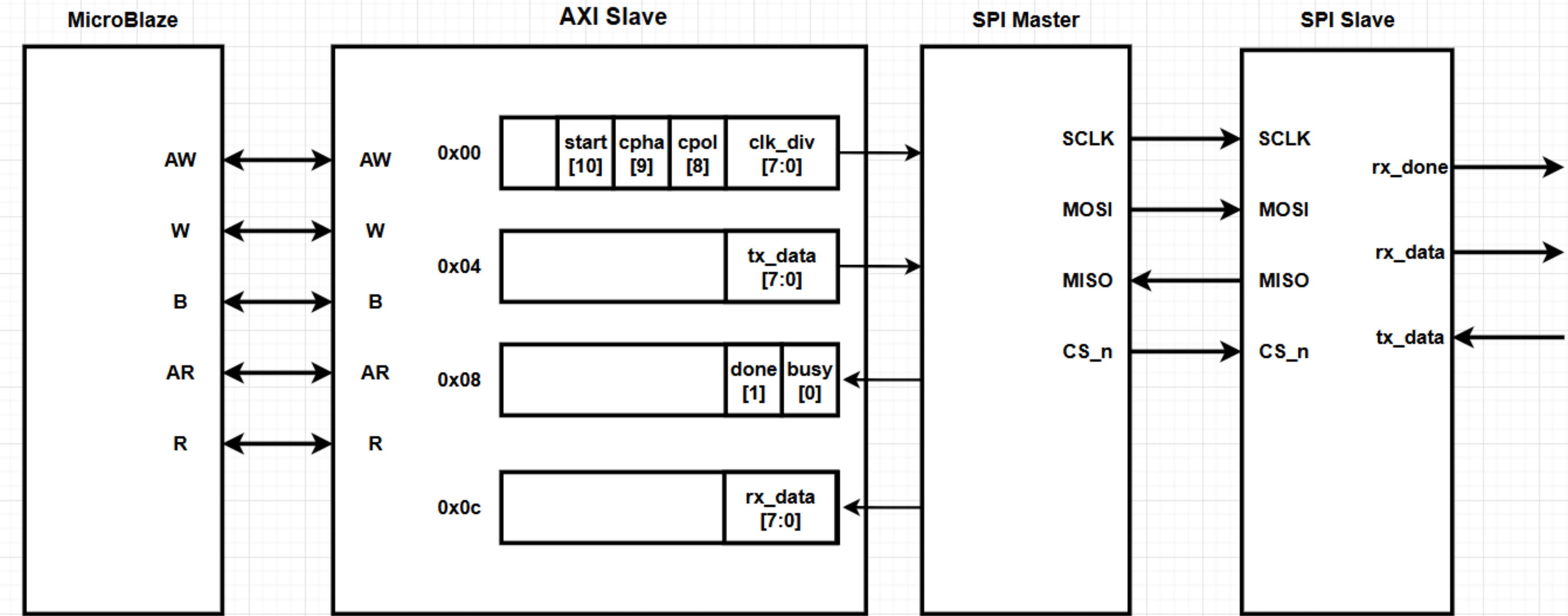
Chapter 4. **Trouble-Shooting**

Chapter 5. **Discussion**

AXI_SPI

MEMORY MAPPING

TMR		0x44A3_0000
CR		
PSC		
ARR		
CNT		
GPIOB		0x44A2_0000
CR		
IDR		
ODR		
SPI		
CONTROL		0x44A1_0000
TX_DATA		0x44A1_0004
STATUS		0x44A1_0008
RX_DATA		0x44A1_000C
GPIOA		0x44A0_0000
CR		
IDR		
ODR		
RAM		0x0000_000U
ROM		



```
always @(*) begin
  case (axi_araddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB])
    2'h0 : reg_data_out <= slv_reg0;
    2'h1 : reg_data_out <= slv_reg1;
    2'h2 : reg_data_out <= {30'b0, done, busy};
    2'h3 : reg_data_out <= {24'b0, rx_data};
    default : reg_data_out <= 0;
  endcase
end
```

```
assign clk_div = slv_reg0[7:0];
assign cpol    = slv_reg0[8];
assign cpha    = slv_reg0[9];
assign start   = slv_reg0[10];
assign tx_data = slv_reg1[7:0];
```

```
logic [7:0] received_count;

spi_slave u_slave (
    .clk(sys_clock),
    .reset(reset),
    .tx_data(sw),
    .rx_data(received_count),
    .done(),
    .busy(),
    .sclk(sclk),
    .mosi(mosi),
    .miso(miso),
    .cs_n(cs_n)
);
```

tx_data & rx_data

- sw에 대한 값을 master로 송신한다.
- 수신받은 rx_data의 값을 led로 보낸다.

```
assign led = received_count;

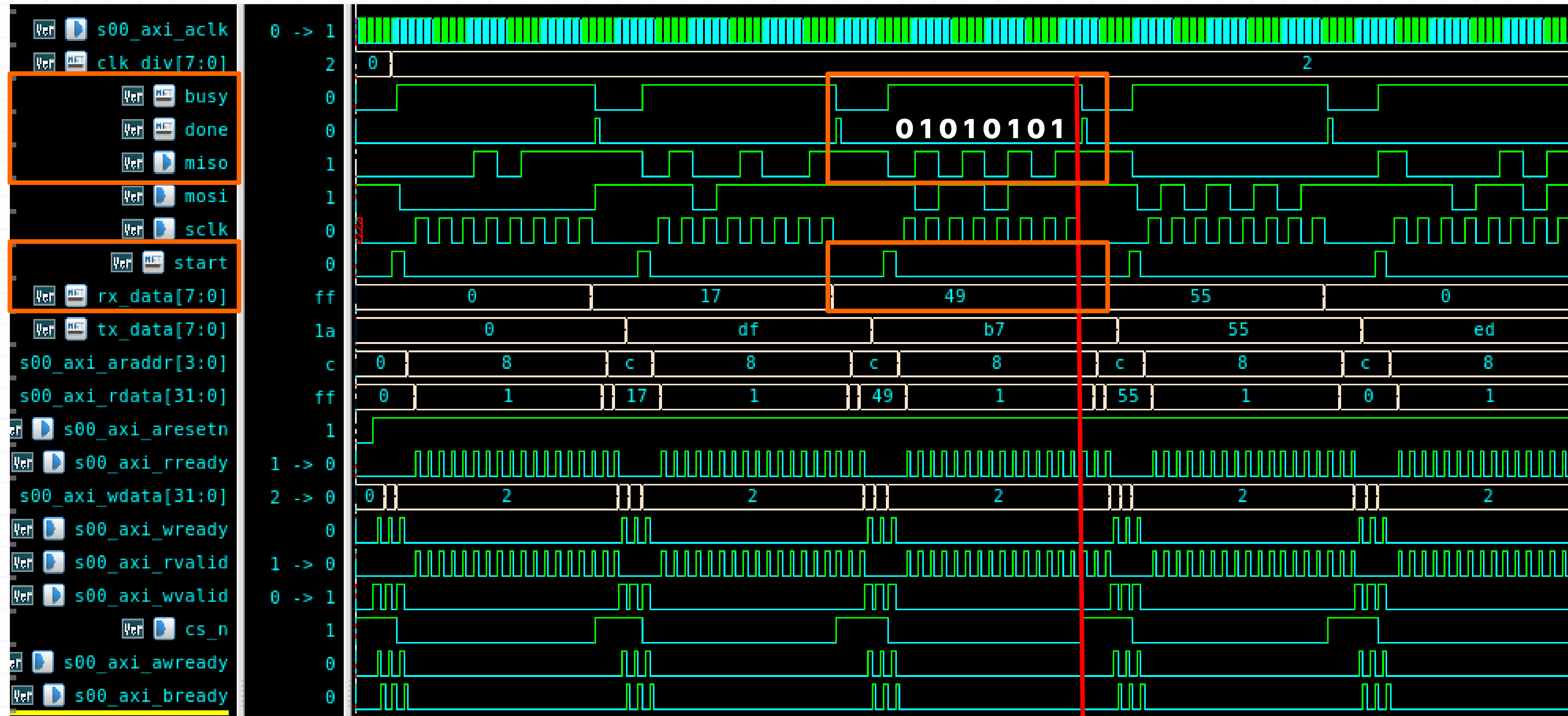
fnd_controller u_fnd_cntr (
    .clk(sys_clock),
    .reset(reset),
    .fnd_in_data({6'b0, received_count}),
    .fnd_digit(fnd_digit),
    .fnd_data(fnd_data)
);
```

```
endmodule
```

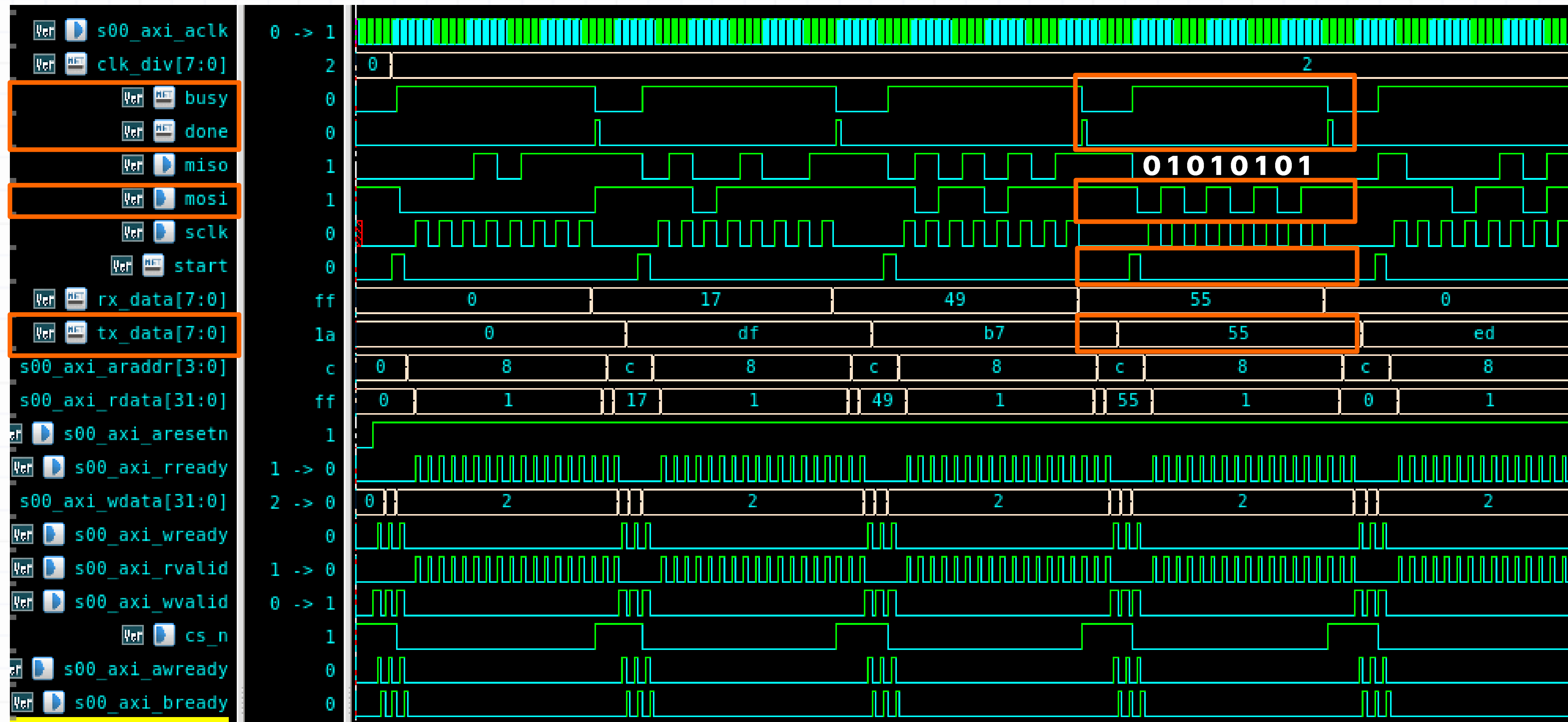
fnd

- 수신받은 8bit의 데이터를 14bit의 fnd_in_data에 넣는다.

- 0x55(01010101)의 miso를 통한 수신과정

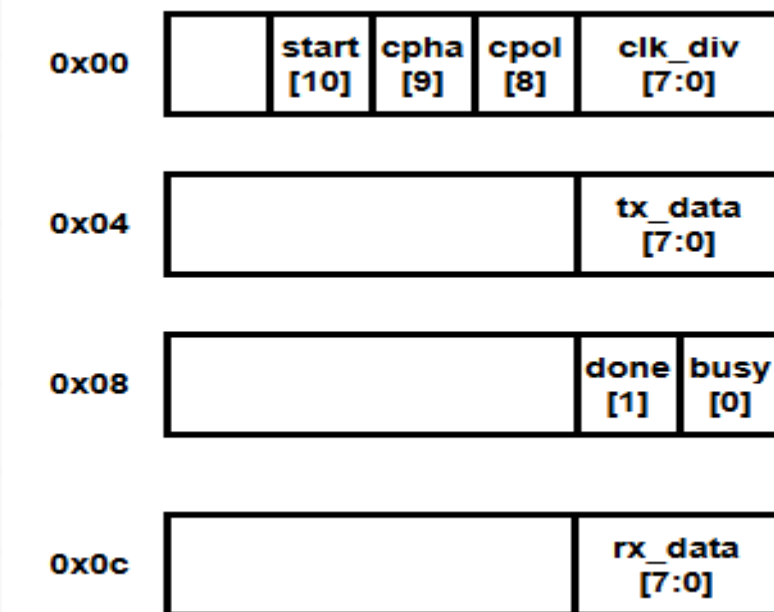
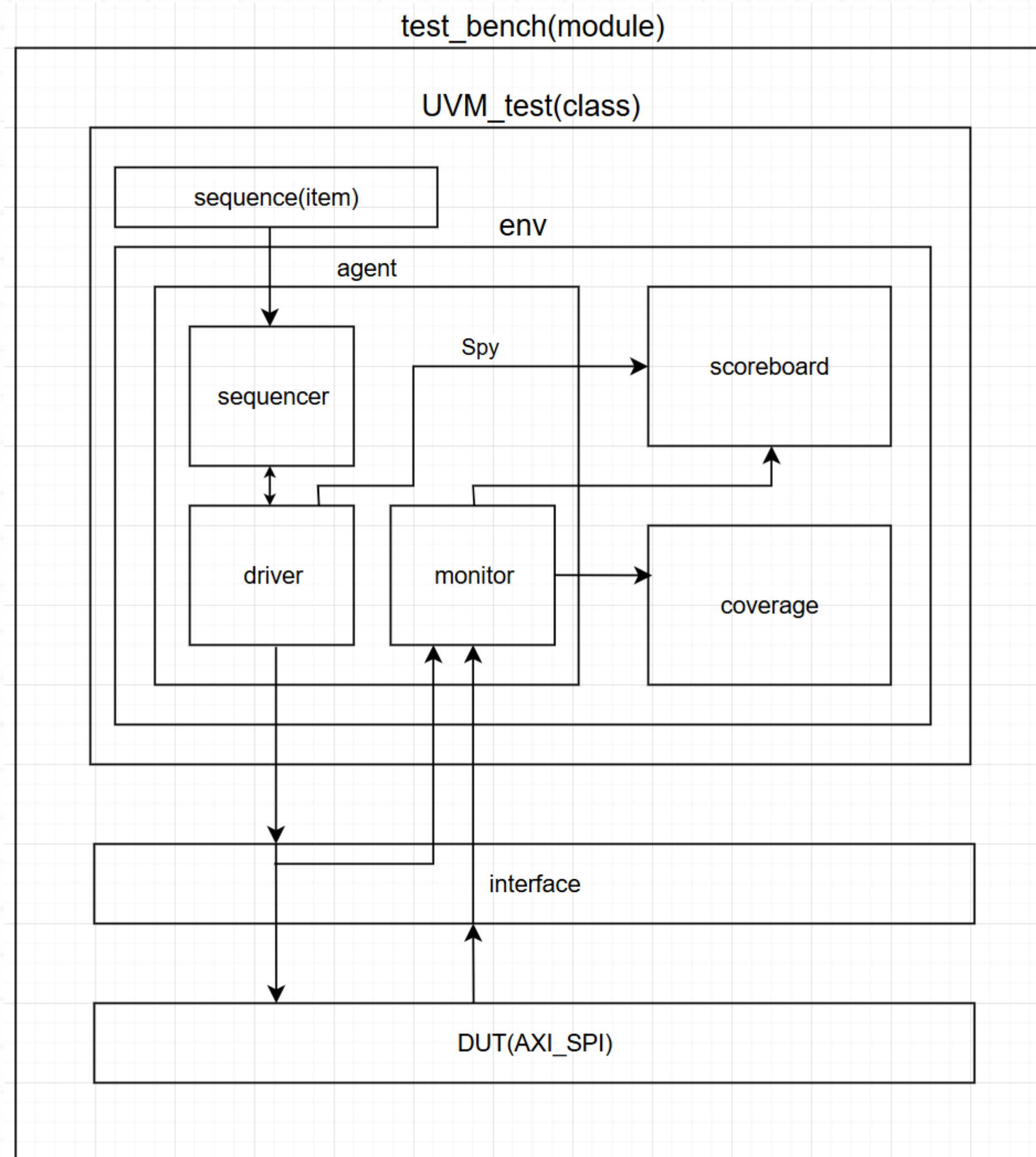


- 0x55(01010101)의 mosi를 통한 송신과정



AXI_SPI_UVM

Chapter 2. AXI_SPI



```
==== AXI Scoreboard Summary ====
Total Checked: 2048
PASS: 2048
FAIL: 0
TEST PASSED: 2048 all matches detected!
```

```
r@seq [spi_axi_rand_seq] 1. WRITE TX REG: [AXI WRITE] ADDR: 0x04 | WDATA: 0x000000aa | SLV_MISO: 0xf7  
[spi_axi_monitor] Monitored AXI WRITE: [AXI WRITE] ADDR: 0x04 | WDATA: 0x000000aa | SLV_MISO: 0xf7  
[spi_axi_scoreboard] Stored Expected RX Data: 0xf7  
r@seq [spi_axi_rand_seq] 2-1. WRITE CTRL (START=1): [AXI WRITE] ADDR: 0x00 | WDATA: 0x00000402 | SLV_MISO: 0xf7  
[spi_axi_monitor] Monitored AXI WRITE: [AXI WRITE] ADDR: 0x00 | WDATA: 0x00000402 | SLV_MISO: 0xf7  
r@seq [spi_axi_rand_seq] 2-2. WRITE CTRL (START=0): [AXI WRITE] ADDR: 0x00 | WDATA: 0x00000002 | SLV_MISO: 0xf7  
[spi_axi_monitor] Monitored AXI WRITE: [AXI WRITE] ADDR: 0x00 | WDATA: 0x00000002 | SLV_MISO: 0xf7  
[spi_axi_monitor] Monitored AXI READ: [AXI READ ] ADDR: 0x08 | RDATA: 0x00000001 | SLV_MISO: 0xxx  
[spi_axi_monitor] Monitored AXI READ: [AXI READ ] ADDR: 0x08 | RDATA: 0x00000001 | SLV_MISO: 0xxx  
[spi_axi_monitor] Monitored AXI READ: [AXI READ ] ADDR: 0x08 | RDATA: 0x00000001 | SLV_MISO: 0xxx  
[spi_axi_monitor] Monitored AXI READ: [AXI READ ] ADDR: 0x08 | RDATA: 0x00000001 | SLV_MISO: 0xxx  
[spi_axi_monitor] Monitored AXI READ: [AXI READ ] ADDR: 0x08 | RDATA: 0x00000001 | SLV_MISO: 0xxx  
[spi_axi_monitor] Monitored AXI READ: [AXI READ ] ADDR: 0x08 | RDATA: 0x00000001 | SLV_MISO: 0xxx  
[spi_axi_monitor] Monitored AXI READ: [AXI READ ] ADDR: 0x08 | RDATA: 0x00000001 | SLV_MISO: 0xxx  
[spi_axi_monitor] Monitored AXI READ: [AXI READ ] ADDR: 0x08 | RDATA: 0x00000001 | SLV_MISO: 0xxx  
[spi_axi_monitor] Monitored AXI READ: [AXI READ ] ADDR: 0x08 | RDATA: 0x00000001 | SLV_MISO: 0xxx  
[spi_axi_monitor] Monitored AXI READ: [AXI READ ] ADDR: 0x08 | RDATA: 0x00000001 | SLV_MISO: 0xxx  
[spi_axi_monitor] Monitored AXI READ: [AXI READ ] ADDR: 0x08 | RDATA: 0x00000001 | SLV_MISO: 0xxx  
[spi_axi_monitor] Monitored AXI READ: [AXI READ ] ADDR: 0x08 | RDATA: 0x00000001 | SLV_MISO: 0xxx  
[spi_axi_monitor] Monitored AXI READ: [AXI READ ] ADDR: 0x08 | RDATA: 0x00000001 | SLV_MISO: 0xxx  
[spi_axi_monitor] Monitored AXI READ: [AXI READ ] ADDR: 0x08 | RDATA: 0x00000001 | SLV_MISO: 0xxx  
[spi_axi_monitor] Monitored AXI READ: [AXI READ ] ADDR: 0x08 | RDATA: 0x00000001 | SLV_MISO: 0xxx  
r@seq [spi_axi_rand_seq] 3. STATUS REG Polling Complete! (busv=0)  
[spi_axi_monitor] Monitored AXI READ: [AXI READ ] ADDR: 0x0c | RDATA: 0x000000f7 | SLV_MISO: 0xxx  
[spi_axi_scoreboard] Match!! Master Read RX: 0xf7 (Expected: 0xf7)  
r@seq [spi_axi_rand_seq] 4. READ RX REG: [AXI READ ] ADDR: 0x0c | RDATA: 0x000000f7 | SLV_MISO: 0xf7
```

```
constraint c_m_tx_data_corner {  
    if (is_write && (addr == ADDR_TX)) {  
        wdata[7:0] dist {  
            8'h00 := 1,  
            8'hFF := 1,  
            8'haa := 1,  
            8'h55 := 1,  
            [8'h01:8'hFE] :/ 6  
        };  
        wdata[31:8] == 24'h0;  
    }  
}
```

- AXI의 wdata는 32bit인 반면 SPI의 tx_data는 8bit이므로 상위 26bit를 0을 넣음.

```
constraint c_s_tx_data_corner {  
    s_tx_data dist {  
        8'h00 := 1,  
        8'hFF := 1,  
        8'haa := 1,  
        8'h55 := 1,  
        [8'h01:8'hFE] :/ 6  
    };  
}
```

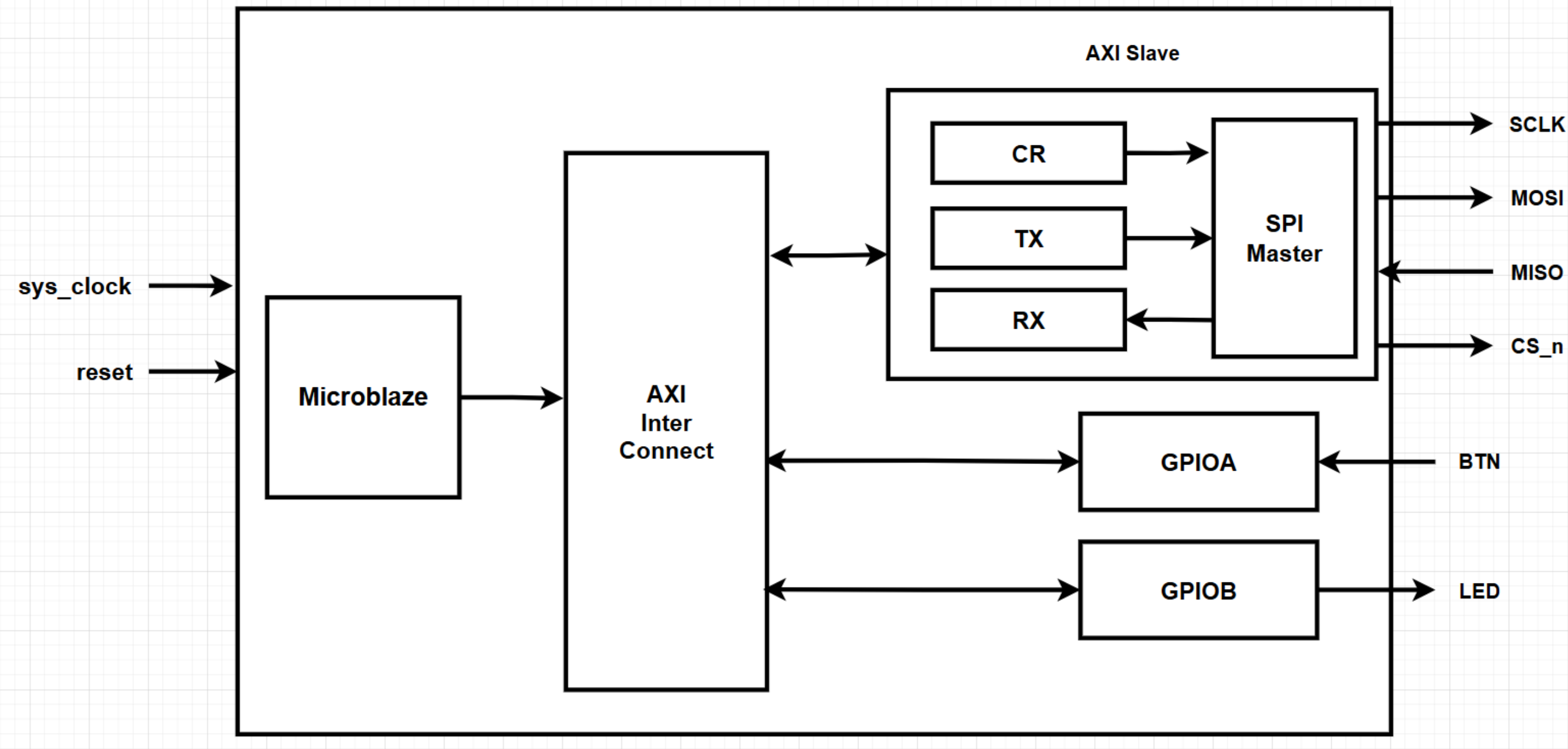
Cover Group Item	Score	Instances	U+C	U	C	X	Goal
\$unit::spi_axi_co...	100.00%		16	0	16	0	100%
cp_m_tx_data	100.00%		8	0	8	0	100%
cp_s_tx_data	100.00%		8	0	8	0	100%

Status	Bin Name	Type	At Least	Size	Hit Count
✓	alt_01	User	1	1	163
✓	alt_10	User	1	1	195
✓	high	User	1	1	423
✓	low	User	1	1	447
✓	lsb_only	User	1	1	1
✓	mid	User	1	1	829
✓	msb_only	User	1	1	2
✓	zero	User	1	1	182

Cover Group Item	Score	Instances	U+C	U	C	X	Goal
\$unit::spi_axi_co...	100.00%		16	0	16	0	100%
cp_m_tx_data	100.00%		8	0	8	0	100%
cp_s_tx_data	100.00%		8	0	8	0	100%

Status	Bin Name	Type	At Least	Size	Hit Count
✓	alt_01	User	1	1	148
✓	alt_10	User	1	1	177
✓	high	User	1	1	422
✓	low	User	1	1	444
✓	lsb_only	User	1	1	3
✓	mid	User	1	1	833
✓	msb_only	User	1	1	5
✓	zero	User	1	1	195

AXI_SPI_C



```
uint8_t SPI_Transfer(uint8_t data) {
    SPI0->TX_DATA = data;
    SPI0->CONTROL = CLK_DIV_VAL | (1 << 10);
    while ((SPI0->STATUS & 0x01) == 0);

    SPI0->CONTROL = CLK_DIV_VAL;
    while ((SPI0->STATUS & 0x01) != 0);

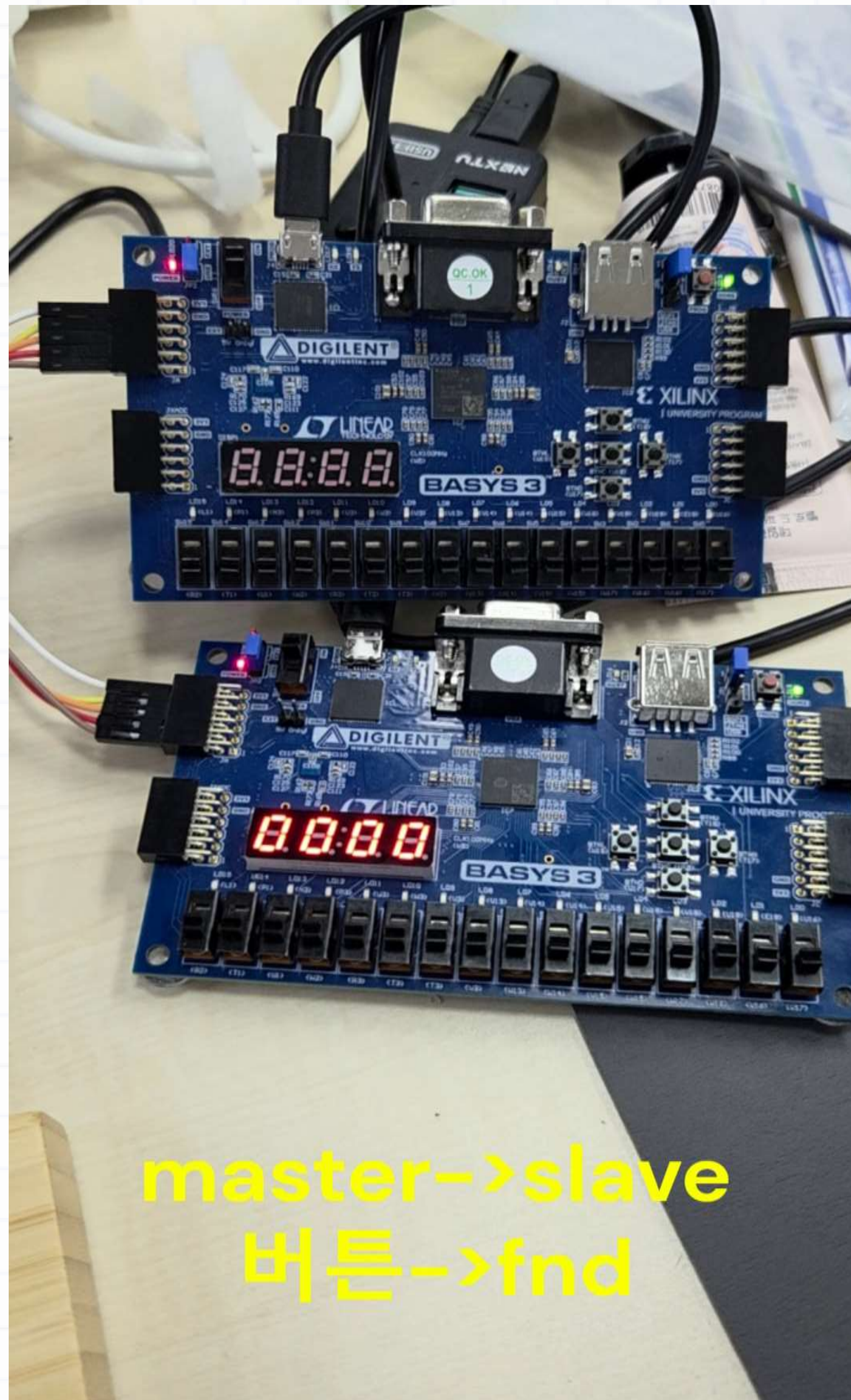
    return (uint8_t)(SPI0->RX_DATA);
}
```

SPI.c

- Start신호를 1로 올렸다 내리며 송수신을 진행한다.
- Busy의 값을 확인하여 통신을 진행한다.

Application	Read / Write		
Driver	Button	LED	SW
HAL	GPIO	TMR	SPI
HW	Button	LED	FND

CONTROL		start [10]	cpha [9]	cpol [8]	clk_div [7:0]
STATUS					done [1]
					busy [0]



SPI

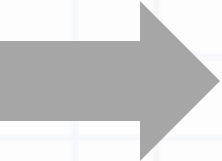
1. Master에서 버튼을 눌러 Slave의 fnd의 숫자를 증가시킨다.
2. Slave에서 스위치를 눌러 Master의 led를 1로 켜다.

AXI_SPI

Trouble-shooting

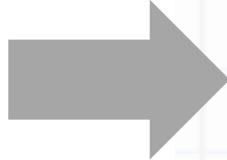
```
if (sclk_rise) begin
    rx_shift_reg <= {rx_shift_reg[6:0], mosi_in};
end

if (sclk_fall) begin
    if (bit_cnt == 3'd0) begin
        done <= 1'b1;
        rx_data <= {rx_shift_reg[6:0], mosi_in};
    end
    bit_cnt <= bit_cnt - 3'd1;
end
```



```
if (sclk_rise) begin
    rx_shift_reg <= {rx_shift_reg[6:0], mosi_in};
end

if (sclk_fall) begin
    if (bit_cnt == 3'd0) begin
        done <= 1'b1;
        rx_data <= rx_shift_reg;
    end
    bit_cnt <= bit_cnt - 3'd1;
end
```



```
end else if (cs_n_rise) begin
    busy <= 1'b0;
    done <= 1'b1;
    rx_data <= rx_shift_reg;
end
```

0000_0001	{0000_001, 1}	0000_0011
0000_0010	{0000_010, 0}	0000_0100
0000_0011	{0000_011, 1}	0000_0111
0000_0100	{0000_100, 0}	0000_1000
0000_0101	{0000_101, 1}	0000_1011

- master에서 8번째 데이터를 보낸 후 1 clk만에 CS를 1로 올림
- > 8번째 데이터를 slave가 읽기 전에 통신 종료 (race condition발생)

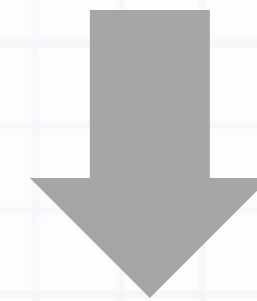


- 안전하게 CS가 올라간 후 rx_data를 업데이트

```
==== AXI Scoreboard Summary ====
Total Checked: 2048
PASS: 0
FAIL: 2048
TEST FAILED: 2048 mismatches detected!
```

```
==== AXI Coverage Summary ====
Overall: 50.0%
wdata(m_tx) : 100.0%
s_tx_data   : 0.0%
==== AXI Coverage Summary ====
```

```
@@seq [spi_axi_rand_seq] 3. STATUS REG Polling Complete! (busy=0)
[spi_axi_monitor] Monitored AXI READ: [AXI READ ] ADDR: 0x0c | RDATA: 0x000000f7 | SLV_MISO: 0xxx
[spi_axi_scoreboard] Mismatch!! Master Read RX: 0xf7 (Expected: 0xxx)
@@seq [spi_axi_rand_seq] 4. READ RX REG: [AXI READ ] ADDR: 0x0c | RDATA: 0x000000f7 | SLV_MISO: 0xf7
```



```
==== AXI Scoreboard Summary ====
Total Checked: 2048
PASS: 2048
FAIL: 0
TEST PASSED: 2048 all matches detected!
```

```
==== AXI Coverage Summary ====
Overall: 100.0%
wdata(m_tx) : 100.0%
s_tx_data   : 100.0%
==== AXI Coverage Summary ====
```

```
r@@seq [spi_axi_rand_seq] 3. STATUS REG Polling Complete! (busy=0)
[spi_axi_monitor] Monitored AXI READ: [AXI READ ] ADDR: 0x0c | RDATA: 0x000000f7 | SLV_MISO: 0xxx
[spi_axi_scoreboard] Match!! Master Read RX: 0xf7 (Expected: 0xf7)
r@@seq [spi_axi_rand_seq] 4. READ RX REG: [AXI READ ] ADDR: 0x0c | RDATA: 0x000000f7 | SLV_MISO: 0xf7
```

```
logic [7:0] miso_spy;
```

```
sif.miso_spy = req.s_tx_data;
```

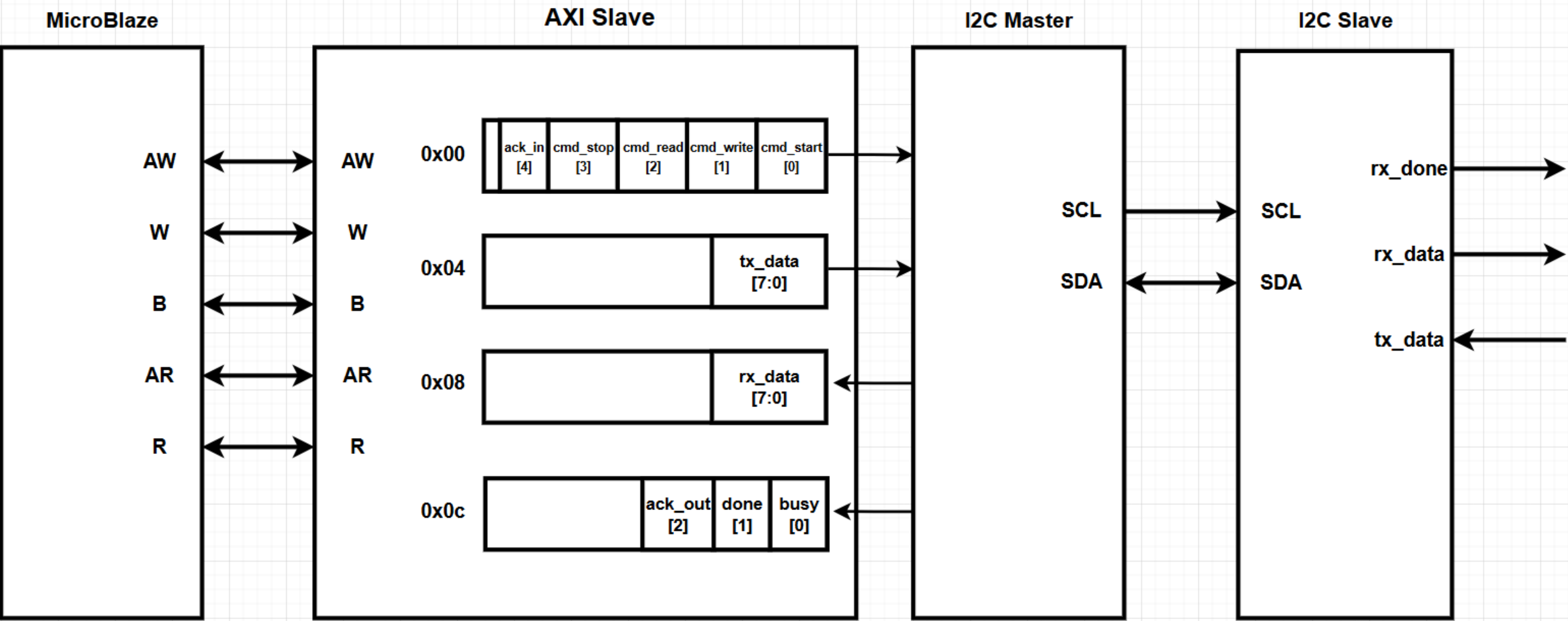
```
wr_item.s_tx_data = sif.miso_spy;
```

```
logic [7:0] exp_rx_q[$];
```

AXI_I2C

MEMORY MAPPING

TMR		0x44A3_0000
CR		
PSC		
ARR		
CNT		
I2C		
CONTROL		0x44A2_0000
TX_DATA		0x44A2_0004
RX_DATA		0x44A2_0008
STATUS		0x44A2_000C
GPIOB		0x44A1_0000
CR		
IDR		
ODR		
GPIOA		0x44A0_0000
CR		
IDR		
ODR		
RAM		0x0000_000U
ROM		



```
case (axi_araddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB])
  2'h0 : reg_data_out <= slv_reg0;
  2'h1 : reg_data_out <= slv_reg1;
  2'h2 : reg_data_out <= {24'b0, rx_data};
  2'h3 : reg_data_out <= {29'b0, ack_out, done_real, busy};
  default : reg_data_out <= 0;
endcase
```

```
assign cmd_start = slv_reg0[0];
assign cmd_write = slv_reg0[1];
assign cmd_read  = slv_reg0[2];
assign cmd_stop  = slv_reg0[3];
assign ack_in    = slv_reg0[4];
assign tx_data   = slv_reg1[7:0];
```

```
logic [7:0] received_count;

I2C_SLAVE u_slave (
    .clk(sys_clock),
    .reset(reset),
    .tx_data(sw),
    .rx_data(received_count),
    .done(),
    .busy(),
    .scl(scl),
    .sda(sda)
);
```

```
assign led = received_count;

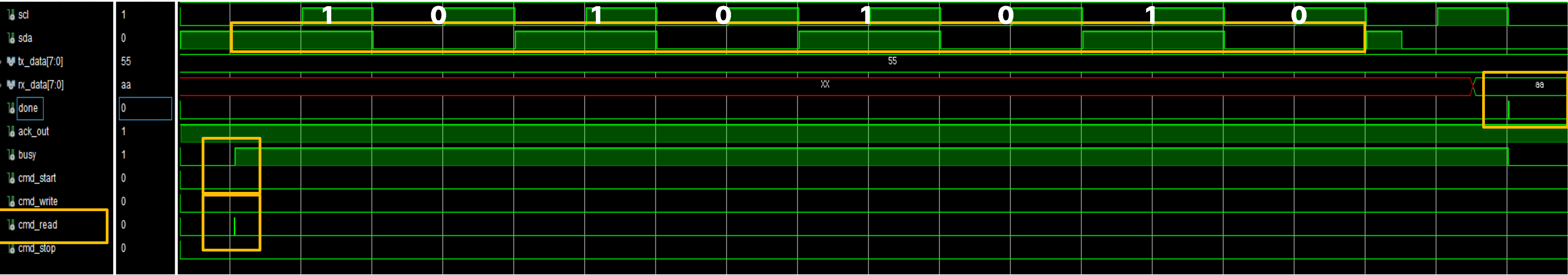
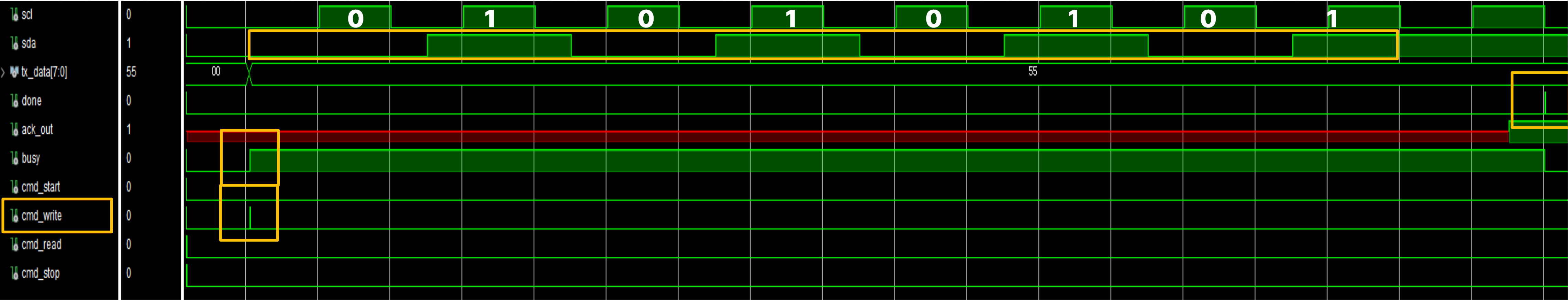
fnd_controller u_fnd_cntr (
    .clk(sys_clock),
    .reset(reset),
    .fnd_in_data({6'b0, received_count}),
    .fnd_digit(fnd_digit),
    .fnd_data(fnd_data)
);
```

tx_data & rx_data

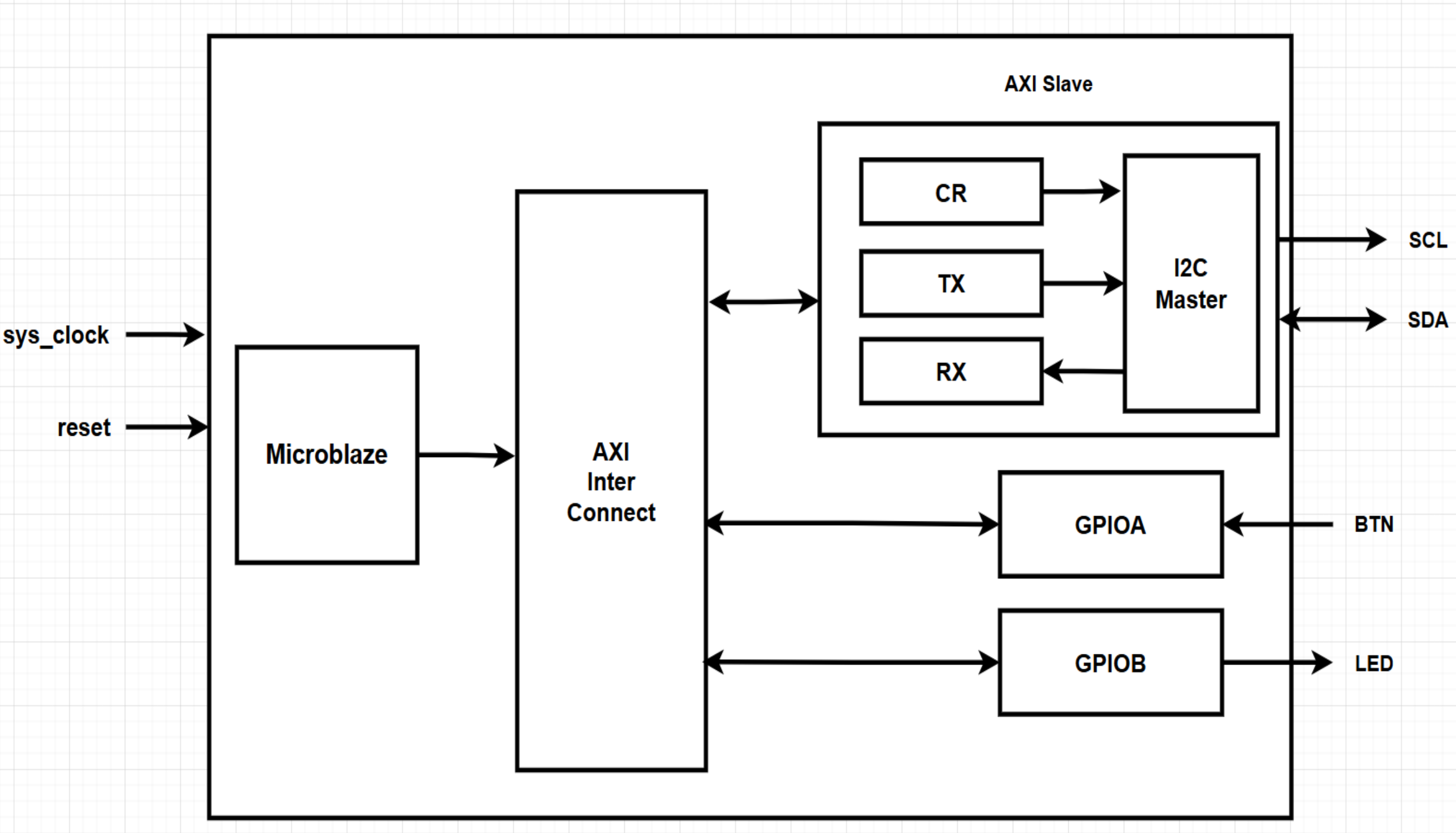
- sw에 대한 값을 master로 송신한다.
- 수신받은 rx_data의 값을 led로 보낸다.

fnd

- 수신받은 8bit의 데이터를 14bit의 fnd_in_data에 넣는다.



AXI_I2C_C



```
void I2C_Start() {
    while (I2C0->STATUS & I2C_STS_BUSY);

    I2C0->CONTROL = I2C_CMD_START;

    while ((I2C0->STATUS & I2C_STS_DONE) == 0);
}

void I2C_Stop() {
    while (I2C0->STATUS & I2C_STS_BUSY);
    I2C0->CONTROL = I2C_CMD_STOP;
    while ((I2C0->STATUS & I2C_STS_DONE) == 0);
}
```

```
uint8_t I2C_Write(uint8_t data) {
    while (I2C0->STATUS & I2C_STS_BUSY);

    I2C0->TX_DATA = data;
    I2C0->CONTROL = I2C_CMD_WRITE;

    while ((I2C0->STATUS & I2C_STS_DONE) == 0);

    return (I2C0->STATUS & I2C_STS_ACK) ? 1 : 0;
}

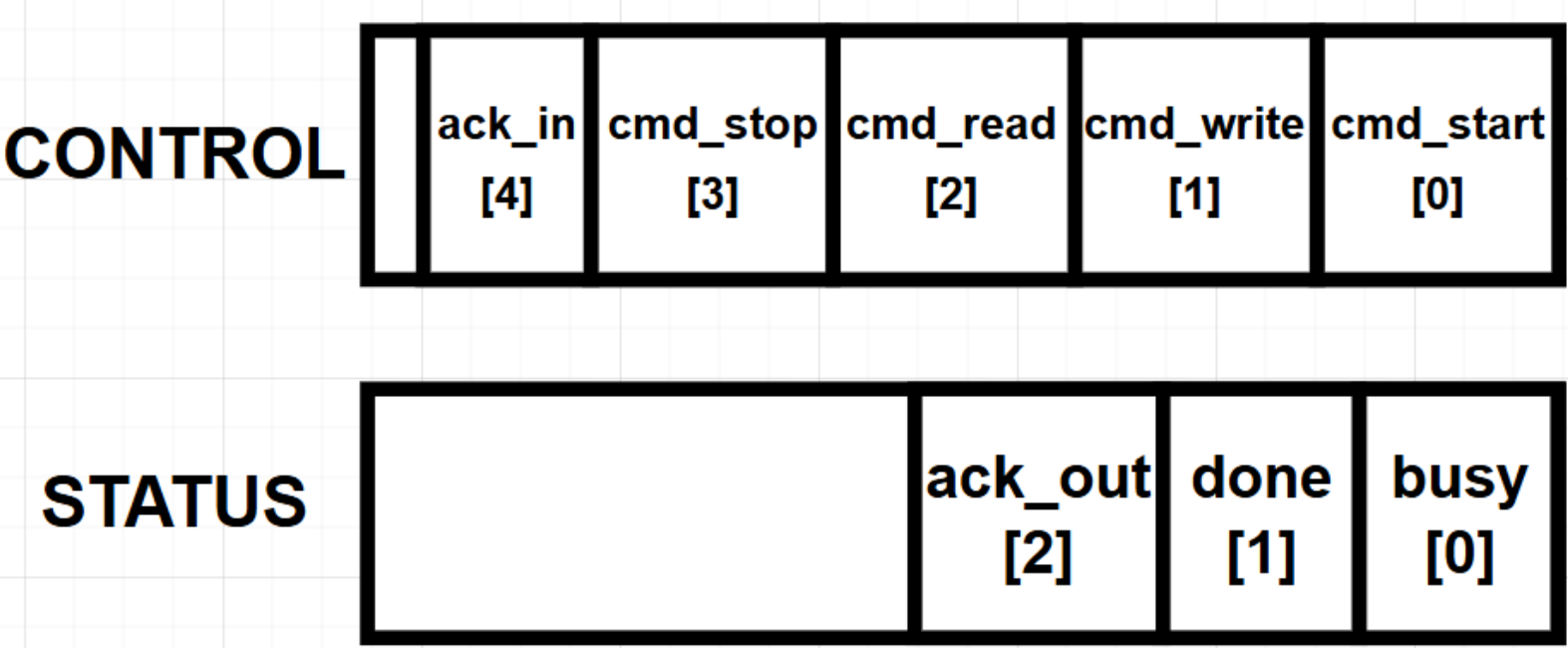
uint8_t I2C_Read(uint8_t ack) {
    while (I2C0->STATUS & I2C_STS_BUSY);

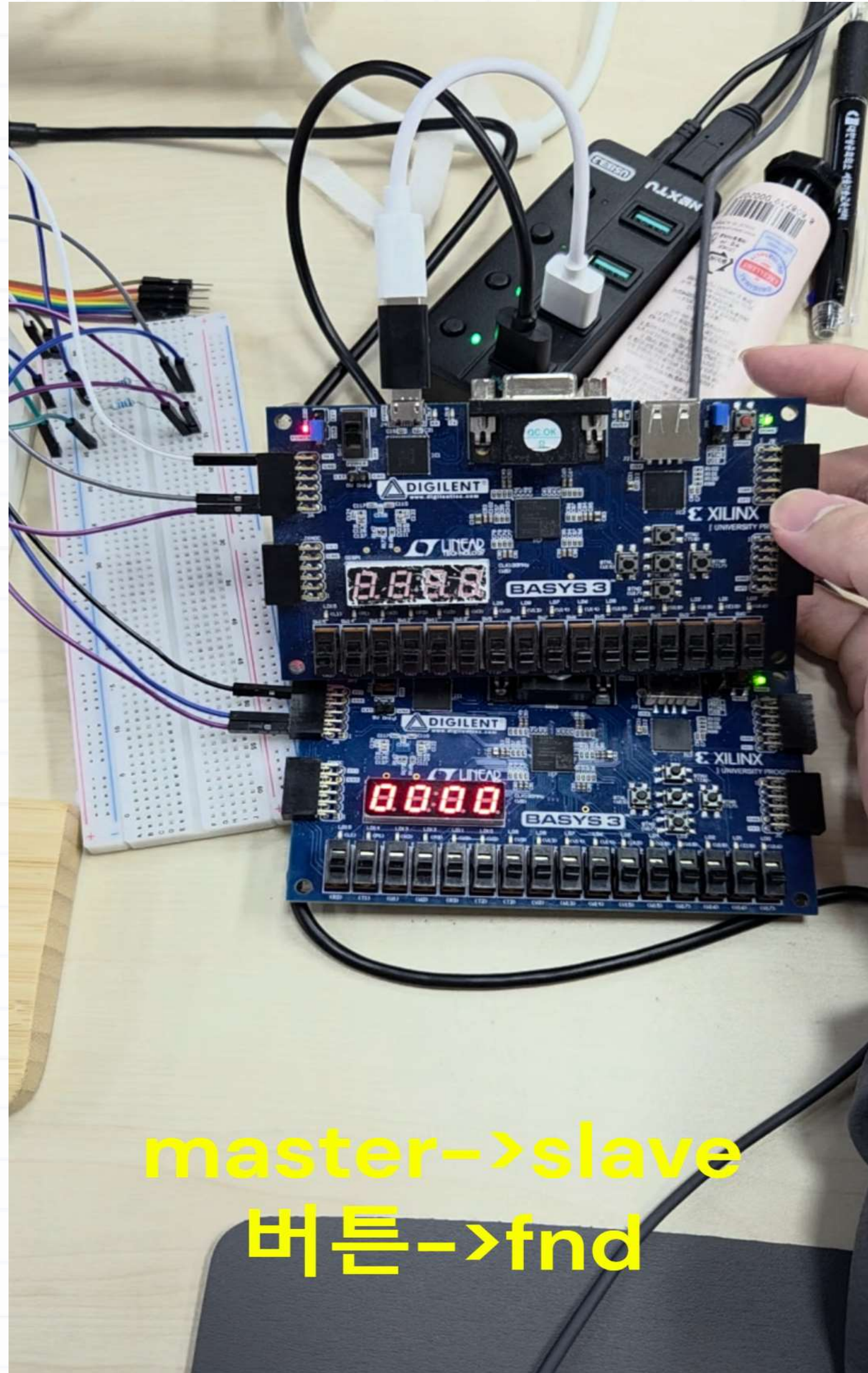
    if (ack == 1) {
        I2C0->CONTROL = I2C_CMD_READ | I2C_ACK_IN;
    } else {
        I2C0->CONTROL = I2C_CMD_READ;
    }

    while ((I2C0->STATUS & I2C_STS_DONE) == 0);

    return (uint8_t)(I2C0->RX_DATA);
}
```

Application	Read / Write			
Driver	Button	LED	SW	
HAL	GPIO		TMR	I2C
HW	Button	LED	SW	FND





I2C

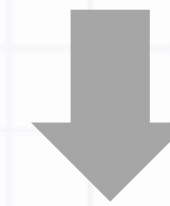
1. Master에서 버튼을 눌러 Slave의 fnd의 숫자를 증가시킨다.
2. Slave에서 스위치를 눌러 Master의 led를 1로 켜다.

AXI_I2C

Trouble-shooting

```
always @(posedge S_AXI_ACLK) begin
    if (S_AXI_ARESETN == 1'b0) begin
        slv_reg0 <= 0;
        slv_reg1 <= 0;
        slv_reg2 <= 0;
        slv_reg3 <= 0;
    end else begin
        slv_reg0[3:0] <= 4'b0000;
        if (slv_reg_wren) begin
            case (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB])
                2'h0:
                    for (
                        byte_index = 0;
                        byte_index <= (C_S_AXI_DATA_WIDTH / 8) - 1;
                        byte_index = byte_index + 1
                    )
                        if (S_AXI_WSTRB[byte_index] == 1) begin
                            // Respective byte enables are asserted as per write strobes
                            // Slave register 0
                            slv_reg0[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];
                        end
            end case
        end
    end
end
```

- 1 클럭 tick 발생 후 done 신호가 0이 됨.



- Done이 1이 되었을 때 유지를 해주다
CPU가 CONTROL 레지스터에 새 명령을 쓰면 0으로 바꿈.

```
always @(posedge S_AXI_ACLK) begin
    if (S_AXI_ARESETN == 1'b0) begin
        done_real <= 1'b0;
    end else begin
        if (done) begin
            done_real <= 1'b1;
        end else if (slv_reg_wren && (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == 2'h0)) begin
            done_real <= 1'b0;
        end
    end
end
```

Discussion

AXI

- AXI의 valid, ready의 핸드셰이크를 이해하게 되었고 APB부터 AXI까지의 버스 프로토콜에 대해 공부하며 데이터의 이동과정에 대해 자세히 알게 됨.

I2C

- tick으로 발생하는 신호들을 어떻게 소프트웨어에게 인식시킬까에 대한 고민을 많이 해보게 되었고 풀업저항을 연결하여 보드에 동작시키는 과정에서 i2c의 이해를 높일 수 있었음

SPI

- mosi와 miso로 통신이 되는 과정에 대해서 자세히 알게 되었고 통신과정에서의 하위비트와 상위비트를 다루는 것에 대해서 많이 배우게 되었음

C

- 하드웨어뿐 아니라 소프트웨어적인 구현으로 칩을 설계해보며 구조체와 포인터에 대해서 더 자세히 알게 되었고 Memory mapped I/O의 필요성과 중요성에 대해서 깨닫게 됨.